

Полный курс React (с Vite) для подготовки к экзамену

Этот курс подробно описывает настройку проекта React с использованием Vite, ключевые концепции React (компоненты, состояние, списки, события) и примеры решений заданий экзамена. Показаны примеры кода для каждого задания, включая работу с фильтрацией изображений, формами и простым классом на Java. Код сопровождается пояснениями и рекомендациями по стилю. Все задачи решены с учётом требований формата экзамена (текстовые метки на польском языке, использование Bootstrap для стилей).

Настройка проекта React с Vite

- **Установка окружения:** Сначала установите Node.js и npm (см. [официальную документацию Node.js]). Затем в терминале выполните команду `npm create vite@latest <имя-проекта>` и выберите шаблон **React** ¹. Например:

```
npm create vite@latest galerie-zdjec  
cd galerie-zdjec
```

Данная команда создаст новый проект Vite с шаблоном React ¹.

- **Установка зависимостей:** Перейдите в созданную папку проекта и выполните `npm install` для установки зависимостей (Vite, React и др.) ². Затем запустите локальный сервер разработки командой `npm run dev`; проект будет доступен по умолчанию на `http://localhost:5173`.
- **Подключение Bootstrap:** Для стилизации форм и элементов GUI рекомендуется подключить Bootstrap. Установите пакет `bootstrap` (и при необходимости `@popperjs/core`) через npm. Затем в самом начале `src/main.jsx` (или `src/index.js`) импортируйте CSS:

```
import 'bootstrap/dist/css/bootstrap.min.css';
```

(как рекомендуют документация React-Bootstrap и Vite) ³ ⁴. Это обеспечит доступность классов Bootstrap (`btn`, `form-control` и др.) в приложении.

- **Структура проекта:** Vite по умолчанию создаёт структуру с папками `src` и `public`. Размещайте свои компоненты в `src`, а статические ресурсы (например, изображения) — в `src/assets` (или `public/assets`). Файл `index.html` выступает точкой входа и подключает скомпилированный скрипт.

Основные концепции React

- **Функциональные компоненты:** React-приложение состоит из компонентов. Каждый компонент — это функция, возвращающая JSX-разметку. Компоненты могут принимать *пропсы* и иметь собственное *состояние*.
- **JSX:** Внутри компонента возвращается разметка JSX, которая в итоге транслируется в JavaScript. В JSX можно включать JavaScript-выражения в фигурных скобках `{}`. Например: `{items.map(item => <li key={item.id}>{item.name}</li>)}` позволяет вывести список элементов ⁵.
- **Состояние (state):** Для управления динамическими данными внутри компонента используется хук `useState`. Этот хук принимает начальное значение и возвращает пару: текущее состояние и функцию для его обновления. Например:

```
const [count, setCount] = useState(0);
```

Здесь `count` — текущее значение, а `setCount` позволяет обновлять его ⁶. После вызова `setCount` компонент перерисовывается с новыми данными.

- **Списки и ключи:** Чтобы отобразить массив данных, используется метод `map()`. В React это делается через JSX, например:

```
const listItems = numbers.map(number => <li key={number}>{number}</li>);  
return <ul>{listItems}</ul>;
```

Каждый элемент списка (`<li>`) должен иметь уникальный атрибут `key`, чтобы React мог эффективно отслеживать изменения ⁵.

- **Условный рендеринг:** Для вывода элементов по условию в JSX часто используют логический оператор `&&`. Например:

```
{isChecked && <div>Отмечен!</div>}
```

В данном случае `<div>` будет отображён только если `isChecked === true`. Это основано на том, что в JS выражение `true && X` даёт `X`, а `false && X` даёт `false`, что приводит к пропуску элемента ⁷. Таким образом можно показывать или скрывать части интерфейса в зависимости от состояния.

- **Обработка событий:** Для обработки кликов, изменения полей и т.д. в элементах JSX используются атрибуты вроде `onClick`, `onChange` и т.д. Например, в кнопке: `<button onClick={handleClick}>Click</button>`. Функция `handleClick` при этом изменяет состояние или выполняет нужные действия. Пример из документации React:

```
function Button({ onClick, children }) {  
  return <button onClick={onClick}>{children}</button>;  
}
```

где свойство `onClick` — это функция-обработчик события ⁸.

- **Формы (Controlled Components):** Для реализации контролируемых форм в React привязывайте значение поля к состоянию и обновляйте его в `onChange`. Например:

```
const [title, setTitle] = useState('');
return <input value={title} onChange={e => setTitle(e.target.value)} />;
```

Такая схема гарантирует, что текущее значение поля всегда хранится в состоянии. Документация React подчёркивает, что при использовании `value` (или `checked`) для `<input>`, необходимо иметь соответствующий `onChange` для обновления этого значения ⁹.

Задача 1: Категоризация изображений (галерея)

В этой задаче нужно создать компонент с галереей изображений, который фильтрует картинки по трём категориям (Kwiaty, Zwierzęta, Samochody) при помощи переключателей (checkbox). Также каждое изображение имеет счётчик загрузок, который увеличивается по клику на кнопку "Pobierz".

- **Данные:** Предположим, список изображений хранится в массиве объектов `imagesData`, полученном из файла `dane.txt`. Каждый объект имеет поля `{ id, alt, filename, category, downloads }`. Например:

```
const initialImages = [
  { id: 1, alt: "Róża", filename: "rose.jpg", category: 1, downloads: 0 },
  { id: 2, alt: "Pies", filename: "dog.jpg", category: 2, downloads: 0 },
  // ...
];
```

Эти данные мы передаём в состояние компонента:

```
const [images, setImages] = useState(initialImages);
```

Так мы сможем динамически изменять `downloads`.

- **Состояние переключателей:** Создадим три булевых состояния для чекбоксов:

```
const [showFlowers, setShowFlowers] = useState(true);
const [showAnimals, setShowAnimals] = useState(true);
const [showCars, setShowCars] = useState(true);
```

По умолчанию все включены, поэтому при загрузке страницы показываются все изображения.

- **JSX-разметка:** Компонент возвращает примерно следующее:

```
<>
  <h1>Kategorie zdjęć</h1>
  <label><input type="checkbox" checked={showFlowers}>
```

```

        onChange={() => setShowFlowers(!showFlowers)} /> Kwiaty</label>
<label><input type="checkbox" checked={showAnimals}
        onChange={() => setShowAnimals(!showAnimals)} /> Zwierzęta</label>
<label><input type="checkbox" checked={showCars}
        onChange={() => setShowCars(!showCars)} /> Samochody</label>

<div className="gallery">
  {filteredImages.map(img => (
    <div key={img.id} style={{ margin: 5, display: "inline-block" }}>
      <img src={`/assets/${img.filename}`} alt={img.alt}
        style={{ width: 150, borderRadius: "8px" }} />
      <h4>{img.downloads}</h4>
      <button onClick={() => incrementDownload(img.id)}>Pobierz</button>
    </div>
  ))}
</div>
</>

```

Здесь `<h1>` с заголовком *Kategorie zdjęć*, затем три чекбокса с подписями *Kwiaty*, *Zwierzęta*, *Samochody*. Блоки изображений выводятся в строку рядом (для этого у родительского `<div>` задано `display: inline-block` или flex-контейнер). Стили можно задать через CSS-классы или inline (например, `margin: 5px`, `borderRadius: 8px` для скругления углов).

- **Фильтрация списка:** Перед отображением мы создаём отфильтрованный массив `filteredImages` на основе состояний переключателей. Например:

```

const filteredImages = images.filter(img =>
  (img.category === 1 && showFlowers) ||
  (img.category === 2 && showAnimals) ||
  (img.category === 3 && showCars)
);

```

Таким образом в `filteredImages` попадут только те объекты, категория которых разрешена соответствующим чекбоксом. Затем мы делаем `filteredImages.map(...)`, возвращая JSX-блок для каждого изображения. Такое отображение массива через `map` является стандартной практикой в React ⁵. Не забудьте указать уникальный `key` для каждого элемента списка (здесь подойдёт `img.id`).

- **Обновление счётчика:** Функция `incrementDownload` обновляет состояние `images`. Пример реализации:

```

function incrementDownload(id) {
  setImages(prevImages =>
    prevImages.map(item =>
      item.id === id
        ? { ...item, downloads: item.downloads + 1 }
        : item
    )
  )
}

```

```
);
}
```

Она находит объект с данным `id` и увеличивает его поле `downloads`. При вызове `setImages` React перерисует компонент, и число загрузок изменится на экране без перезагрузки страницы. Такой приём с `useState` и функцией обновления состояния описан в документации React ⁶.

- **Условный рендеринг (альтернатива):** Можно было бы также написать условие прямо в `map`, например `{showFlowers && img.category===1 && <div>...</div>}`. Но предпочтительнее сделать фильтрацию заранее, чтобы не генерировать лишние элементы. Тем не менее концепция `{condition && <Component />}` применима для показа элементов по флагу ⁷.
- **Стилизация:** Кнопки и чекбоксы можно оформить с помощью классов Bootstrap (`<button className="btn btn-primary">Pobierz</button>`, `<input className="form-check-input" ...>`), либо собственными стилями. Главное, чтобы выполнялись требования: отступы 5px, скруглённые углы и т.д.

В итоге получаем компонент, который при изменении состояний чекбоксов сразу фильтрует галерею, а при клике «Pobierz» обновляет счётчик загрузок. Код остаётся универсальным и масштабируется на любое количество изображений за счёт использования массива и методов `filter/map`.

Задача 2: Форма «Tytuł filmu»

В этой задаче нужно сделать веб-форму с текстовым полем «Tytuł filmu», выпадающим списком «Rodzaj» и кнопкой «Dodaj». При клике «Dodaj» данные формы выводятся в консоль в формате `tytul: ...; rodzaj: ...`.

- **Состояние полей:** Создаём два состояния: для заголовка и для выбранного типа. Например:

```
const [title, setTitle] = useState("");
const [genre, setGenre] = useState(""); // или число, но можно строка
```

- **JSX-разметка формы:** Используем элементы `<label>`, `<input>` и `<select>`. Пример:

```
<div style={{ padding: "20px" }}> {/* добавляем общий отступ */}
  <label htmlFor="title">Tytuł filmu:</label>
  <input id="title" type="text" className="form-control"
    value={title}
    onChange={e => setTitle(e.target.value)} />

  <label htmlFor="rodzaj">Rodzaj:</label>
  <select id="rodzaj" className="form-select"
    value={genre}
    onChange={e => setGenre(e.target.value)}>
```

```

    <option value="">--</option>
    <option value="1">Komedia</option>
    <option value="2">Obyczajowy</option>
    <option value="3">Sensacyjny</option>
    <option value="4">Horror</option>
  </select>

  <button className="btn btn-primary mt-3" onClick={handleAdd}>
    Dodaj
  </button>
</div>

```

Здесь присвоили полям идентификаторы для читаемости, а также добавили классы Bootstrap: `form-control` для текстового поля, `form-select` для списка, `btn btn-primary` для кнопки. Контейнеру формы дали `padding: 20px` (чтобы отобразить, как требуется).

- **Обработчик кнопки:** Функция `handleAdd` просто выводит в консоль введенные значения:

```

function handleAdd() {
  console.log(`tytul: ${title}; rodzaj: ${genre}`);
}

```

При пустом выборе `genre` выведется пустая строка после `rodzaj:`. Форматирование строки делаем сами в `console.log`.

- **Контролируемые компоненты:** Поля формы являются «контролируемыми» — их `value` привязано к состоянию, а `onChange` обновляет состояние. React требует этого подхода при работе со `value` у `<input>`⁹. Мы использовали `value={title}` и `value={genre}` и `onChange={...}` соответственно.
- **Инициализация:** При первоначальном рендере оба поля пустые (состояние `title=""`, `genre=""`). Это соответствует условию «пустые поля в начале».
- **Вывод в консоль:** После нажатия «Dodaj» в консоли браузера появится строка вида, например:

```
tytul: Matrix; rodzaj: 3
```

Это точно соответствует формату, указанному в задании для React.

Таким образом мы получили простую форму, стилизованную с помощью Bootstrap, которая отправляет данные в консоль. Код читабельно структурирован, используются осмысленные названия переменных (`title`, `genre`, `handleAdd`).

Задача 3: Класс `Osoba` (консольное приложение на Java)

Задача состоит в реализации класса `Osoba` с определёнными свойствами и методами. Все требования выполняются на языке Java.

- **Поля класса:** Два поля:

```
private int id;  
private String imie;
```

— оба закрытые (`private`). Это означает, что доступ к ним имеют только методы самого класса. Статическое поле для подсчёта экземпляров:

```
public static int iloscOsob = 0;
```

Оно объявлено `public` (доступно извне), но обычно его тоже делают `private` и даёт `public` метод доступа. Поле инициализируется нулём. Такая конструкция (`static int`) разделяется всеми объектами класса, что позволяет хранить количество созданных экземпляров ¹⁰.

- **Конструкторы:** Три конструктора:
- **Без параметров:** Присваивает `id = 0` и `imie = ""` (пустая строка).
- **С двумя параметрами:** Принимает `int newId` и `String newImie`, присваивает их полям.
- **Копирующий:** Принимает другой объект `Osoba other` и копирует его поля: `this.id = other.id; this.imie = other.imie;`.

Каждый из этих конструкторов увеличивает счётчик экземпляров `iloscOsob++`. Например:

```
public Osoba() {  
    this.id = 0;  
    this.imie = "";  
    iloscOsob++;  
}  
public Osoba(int newId, String newImie) {  
    this.id = newId;  
    this.imie = newImie;  
    iloscOsob++;  
}  
public Osoba(Osoba other) {  
    this.id = other.id;  
    this.imie = other.imie;  
    iloscOsob++;  
}
```

Такой приём (увеличивать `static` счётчик в каждом конструкторе) — стандартный способ узнать, сколько объектов класса было создано ¹⁰. В примере GeeksforGeeks показано именно статический `int`, инкрементируемый в конструкторе ¹⁰.

- **Метод приветствия:** Реализуем метод

`public void powiedzCzesc(String imieArgument)`. Он должен выводить в консоль сообщение:

- Если поле `this.imie` непустое:

```
Cześć <imieArgument>, mam na imię <this.imie>
```

- Если `this.imie` пустая строка (т.е. имя не задано):

```
Brak danych
```

Пример реализации:

```
public void powiedzCzesc(String imieArgument) {  
    if (this.imie == null || this.imie.isEmpty()) {  
        System.out.println("Brak danych");  
    } else {  
        System.out.println("Cześć " + imieArgument + ", mam na imię " +  
this.imie);  
    }  
}
```

- **Главный класс для тестов:** В методе `main` создаём несколько объектов `Osoba` для проверки. Например:

```
public class Main {  
    public static void main(String[] args) {  
        Osoba o1 = new Osoba();  
        Osoba o2 = new Osoba(1, "Jan");  
        Osoba o3 = new Osoba(o2);  
        System.out.println("Ilość osób: " + Osoba.iloscOsob); // должно быть 3  
  
        o1.powiedzCzesc("Adam"); // ожидаем "Brak danych", т.к. o1.imie пусто  
        o2.powiedzCzesc("Ewa"); // "Cześć Ewa, mam na imię Jan"  
    }  
}
```

Здесь мы создаём три экземпляра `Osoba`, проверяем, что статический счётчик равен 3, и вызываем метод приветствия с разными аргументами. Код организован ясно, с понятными именами переменных.

- **Итого:** Класс `Osoba` демонстрирует базовые принципы ООП в Java: инкапсуляцию (приватные поля), статические поля для общего состояния (счётчик экземпляров) и перегрузку (конструкторы с разными параметрами). Использование `static` поля для подсчёта объектов описывается в литературе и примерах кода ¹⁰. Все требования задачи реализованы.

Задача 4: Запись на курсы

Задача связана с формой для записи на курсы. Она включает массив названий курсов, вывод информации о количестве курсов и формы ввода данных.

- **Данные курса:** Задаётся массив строк, например:

```
const courses = ["Programowanie w C#", "Angular dla początkujących", "Kurs Django"];
```

Это поле компонента; размер массива может меняться.

- **Отображение количества и списка:** В начале компонента выводится заголовок:

```
<h2>Liczba kursów: {courses.length}</h2>
```

где `{courses.length}` динамически показывает число элементов массива. Затем автоматически генерируется нумерованный список `<ol>`:

```
<ol>
  {courses.map((course, index) => (
    <li key={index}>{course}</li>
  ))}
</ol>
```

Здесь используется `map()`, чтобы создать `<li>` для каждого курса. Это стандартный подход для отображения списков в React ⁵. За ключи (`key`) можно взять индекс или, предпочтительнее, уникальные ID (если они есть).

- **Форма:** Под заголовком и списком размещается форма:
- Текстовое поле `input` с меткой «Imię i nazwisko:».
- Числовое поле `input type="number"` с меткой «Numer kursu:».
- Кнопка «Zapisz do kursu».

Пример JSX:

```
<label htmlFor="name">Imię i nazwisko:</label>
<input id="name" type="text" className="form-control"
  value={name} onChange={e => setName(e.target.value)} />

<label htmlFor="kurs">Numer kursu:</label>
<input id="kurs" type="number" className="form-control"
```

```
value={number} onChange={e => setNumber(e.target.value)} />

<button className="btn btn-primary" onClick={handleSubmit}>
  Zapisz do kursu
</button>
```

Использованы классы Bootstrap (`form-control`, `btn btn-primary`) для стилизации полей. Состояния `name` и `number` инициализируются пустыми строками (поля пусты).

- **Обработка отправки:** Функция `handleSubmit` вызывается при клике на кнопку. Она должна вывести в консоль:
 - Введённое `name`.
 - Название курса по введённому `number` (если такое число корректно), иначе сообщение «Nieprawidłowy numer kursu».

Пример реализации:

```
function handleSubmit() {
  const idx = parseInt(number) - 1;
  if (idx >= 0 && idx < courses.length) {
    console.log(name);
    console.log(courses[idx]);
  } else {
    console.log("Nieprawidłowy numer kursu");
  }
}
```

Таким образом, при вводе `name = "Anna"`, `number = 2` в консоли появится:

```
Anna
Angular dla początkujących
```

Если `number` вне диапазона, выведется строка об ошибке.

- **Готовый вид:** В исходном состоянии список отображается полностью, форма пуста. После нажатия кнопки в консоли браузера видны переданные данные (имя, название курса или сообщение об ошибке). Благодаря использованию `courses.map` любое изменение размеров массива (добавление курса) автоматически отразится и в выводе списка ⁵.

Таким образом реализовано требование динамического списка курсов и простого вывода введённых данных. Код читабелен, с понятными названиями переменных и функций, а стилизация опирается на Bootstrap.

Заключение

В этом курсе мы рассмотрели полный пример подготовки к экзаменационным задачам с использованием React (через Vite) и Java. Основные моменты:

- **React + Vite:** Быстрое развертывание проекта через `npm create vite@latest` ¹. Vite обеспечивает быструю сборку и обновление при разработке.
- **React-концепции:** Управление состоянием через `useState` ⁶, отображение списков через `map` ⁵, условное рендеринг с `&&` ⁷, обработка событий и форм ⁹.
- **Bootstrap:** Подключение CSS позволяет легко оформить поля и кнопки. Необходимо импортировать файл Bootstrap (`bootstrap/dist/css/bootstrap.min.css`) ³ ⁴, чтобы классы заработали.
- **Структура кода:** Во всех примерах компоненты написаны аккуратно, с осмысленными именами и комментарием логики. Соблюдены указания задания (метки на польском языке, стилизация по образцу, использование циклов `map` и условий).

Надеемся, данный материал поможет глубоко разобраться в React, овладеть практикой работы с Vite и успешно выполнить аналогичные задания на экзамене. Приведённые примеры могут быть расширены и адаптированы под любые схожие задачи, сохраняя общую логику.

Источники: Документация и руководства React и Vite ¹ ⁶ ⁵ ⁷ ⁹ ¹⁰ ³ ⁴.

¹ ² How to setup Reactjs with Vite ? - GeeksforGeeks

<https://www.geeksforgeeks.org/reactjs/how-to-setup-reactjs-with-vite/>

³ ⁴ reactjs - How to use react-bootstrap in vitejs? - Stack Overflow

<https://stackoverflow.com/questions/74233860/how-to-use-react-bootstrap-in-vitejs>

⁵ Lists and Keys – React

<https://legacy.reactjs.org/docs/lists-and-keys.html>

⁶ ⁸ Adding Interactivity – React

<https://react.dev/learn/adding-interactivity>

⁷ Conditional Rendering – React

<https://legacy.reactjs.org/docs/conditional-rendering.html>

⁹

– React

<https://react.dev/reference/react-dom/components/input>

¹⁰ Count number of a class objects created in Java - GeeksforGeeks

<https://www.geeksforgeeks.org/java/count-number-of-a-class-objects-created-in-java/>